

useMemo Hook

- useMemo is a handy tool in React that helps make your apps faster.
- Imagine you have a function that does some heavy lifting, like calculating a complex math problem or formatting data.
- Normally, React recalculates this function every time your component renders, even if the inputs are the same.
- That can slow things down.
- But with useMemo, React remembers the result of that function as long as its inputs stay the same.
- So, if your inputs don't change, React just grabs the stored result instead of recalculating it every time.
- It helps improve performance by preventing expensive computations from running unless their dependencies change.
- In simple terms, useMemo is like having a smart assistant who remembers the answers to math problems, so you don't have to solve them again and again.

Syntax

```
const memoizedValue = useMemo(() => computeExpensiveValue(a, b), [a, b]);
```

- The first argument is a function that returns the computed value.
- The second argument is an array of dependencies ([a, b]). The function only re-runs when these dependencies change.

Use useMemo when:

1. Computations are expensive – If a function is costly and doesn't need to re-run every render, memoization helps.
2. Avoiding unnecessary re-renders – If a function is recomputing the same output repeatedly, useMemo caches it.
3. Filtering, sorting, and calculating derived state – When working with lists, useMemo prevents unnecessary processing.

Memoizing Expensive Calculations

- Expensive calculations are CPU-intensive operations that slow down the app if re-executed too frequently.

Example 1: Factorial Calculation without useMemo

```
import React, { useState } from 'react'

function Q1() {
  const [num, setNum] = useState(20)
  const [count, setCount] = useState(0)

  const factorial = (n) =>{
    console.log("Computing the factorial ...")
    return n<=1 ? 1: n * factorial(n-1)
  };

  const fact = factorial(num)
  return (
    <div>
      <p>The factorial of {num} is {fact}</p>
      <button onClick={()=>setNum(num + 1)}>Increase Num</button>

      <p>Count: {count}</p>
      <button onClick={()=>setCount(count + 1)}>Increase Count</button>
    </div>
  )
}

export default Q1
```

- The code has a performance issue because the factorial(num) function is called inside the component body, meaning it recalculates on every render, even when num hasn't changed.

- This happens because React re-renders the entire component whenever any state updates, including the count state.
- So, clicking "Increase Count", which has nothing to do with num, still triggers a full recomputation of the factorial.
- This is inefficient, especially for large numbers.
- The solution is to use useMemo to memoize the factorial calculation, ensuring it only recalculates when num changes, thus preventing unnecessary re-renders and improving performance.

Factorial using useMemo

```
import React, { useMemo, useState } from 'react'

function Q1() {
  const [num, setNum] = useState(20)
  const [count, setCount] = useState(0)

  const factorial = (n) => {
    console.log("Computing the factorial ...")
    return n <= 1 ? 1 : n * factorial(n-1)
  };

  const fact = useMemo(() => factorial(num), [num])
  return (
    <div>
      <p>The factorial of {num} is {fact}</p>
      <button onClick={() => setNum(num + 1)}>Increase Num</button>

      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increase Count</button>
    </div>
  )
}
```

```
}
```

```
export default Q1
```

- We can cache the factorial result using useMemo so that it only recomputes when num changes.
- Factorial is only recomputed when num changes.
- Clicking "Increase Count" no longer recalculates factorial.
- Improved performance for large numbers.

Memoizing Filtered Data in a List

- When filtering a list in React, the filter operation is often recalculated on every render, even when the original data hasn't changed.
- This can lead to performance issues, especially if the list is large or the filtering logic is complex.
- Using useMemo, we can cache the filtered list, ensuring that the filtering process only runs when necessary, i.e., when the filtering criteria or the list changes.

Example: Filtering users using Role without useMemo

```
import React from 'react'
import { useState } from 'react'
const users = [
  { id: 1, name: "Cynthia", role: "Admin" },
  { id: 2, name: "Mark", role: "User" },
  { id: 3, name: "Tyler", role: "Admin" },
]

function Q2() {
  const [roleFilter, setRoleFilter] = useState("Admin")
  const filteredUsers = users.filter((user) => {
    console.log("Filtering users...");
    return user.role === roleFilter;
  })
}
```

```

});
return (
  <div>
    <h2>Filtered Users</h2>
    <button onClick={() => setRoleFilter("Admin")}>Admins</button>
    <button onClick={() => setRoleFilter("User")}>Users</button>
    <ul>
      {filteredUsers.map((user) => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
    <p>Testing</p>
  </div>
)
}

```

export default Q2

- Every re-render causes filtering to be recalculated, even when roleFilter hasn't changed.

With useMemo

```

import React from 'react'
import { useState, useMemo } from 'react'
const users = [
  { id: 1, name: "Cynthia", role: "Admin" },
  { id: 2, name: "Mark", role: "User" },
  { id: 3, name: "Tyler", role: "Admin" },
]

function Q2() {
  const [roleFilter, setRoleFilter] = useState("Admin")

```

```

const [count, setCount] = useState(0)

const filteredUsers = useMemo(() => {
  console.log("Filtering users...");
  return users.filter((user) => user.role === roleFilter);
}, [roleFilter]);
return (
  <div>
    <h2>Filtered Users</h2>
    <button onClick={() => setRoleFilter("Admin")}>Admins</button>
    <button onClick={() => setRoleFilter("User")}>Users</button>
    <ul>
      {filteredUsers.map((user) => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
    <p>Count: {count}</p>
    <button onClick={() => setCount(count + 1)}>Increase Count</button>
  </div>
)
}

```

export default Q2

- filteredUsers only updates when roleFilter changes, preventing unnecessary calculations.
- If other states update, the filtering function doesn't run again.
- Better performance, especially for large datasets.